

THUẬT TOÁN THAM LAM

- Sơ đồ chung thuật toán
- Các ví dụ minh họa
 - Bài toán về các đoạn thẳng không giao nhau
 - Mã hóa Huffman

THUẬT TOÁN THAM LAM



- Các thuật toán tham lam là một cách tiếp cận trực tiếp để xây dựng thuật toán giải nhiều lớp bài toán khác nhau.
- Chúng ta sẽ đưa ra quyết định dựa ngay vào thông tin đang có, và trong tương lai sẽ không xem xét lại tác động của các quyết định đã nhận trong quá khứ.
- Vì vậy, các thuật toán dạng này rất **dễ đề xuất**, và thông thường chúng **không đòi hỏi nhiều thời gian tính**. Tuy nhiên, các thuật toán dạng này **thường không cho kết quả đúng**.

SƠ ĐỒ CHUNG



- Thuật toán tham lam dùng để **giải quyết các bài toán tối ưu**: Tìm lời giải với giá trị hàm mục tiêu lớn nhất (hoặc nhỏ nhất)
- Lời giải cần tìm có thể mô tả như là bộ gồm hữu hạn các thành phần thoả mãn các điều kiện nhất định
- **C là tập các ứng cử viên** có thể lựa chọn làm các thành phần của lời giải

SƠ ĐỒ CHUNG



- Xuất phát từ lời giải rỗng, thuật toán xây dựng lời giải của bài toán theo từng bước, ở mỗi bước sẽ chọn một phần tử từ tập ứng cử viên và bổ sung vào lời giải hiện có.
- Hàm **Solution(S)** nhận biết tính chấp nhận được của lời giải S.
- Hàm **Select(C)** chọn từ tập C ứng cử viên có triển vọng nhất để bổ sung vào lời giải hiện có.
- Hàm **Feasible($S \cup x$)** kiểm tra tính chấp nhận được của lời giải bộ phận $S \cup x$.

SƠ ĐỒ CHUNG



procedure Greedy;

(* Giả sử C là tập các ứng cử viên *)

begin

S := $\emptyset$; (* S lời giải xây dựng theo thuật toán *)

while (C $\neq \emptyset$) **and not** Solution(S) **do**

begin

x $\leftarrow$ **Select**(C);

C := C \ x;

if Feasible (S $\cup$ x) **then** S := S $\cup$ x ;

end;

if Solution(S) **then return** S;

end;

CHỨNG MINH TÍNH ĐÚNG ĐẴN



- Để chỉ ra thuật toán không đúng đắn chỉ cần đưa ra một phản ví dụ (một bộ dữ liệu mà đối với nó thuật toán không cho lời giải đúng)
- Chứng minh tính đúng đắn của thuật toán khó hơn nhiều.

LẬP LUẬN BIẾN ĐỔI (EXCHANGE ARGUMENT)



- Giả sử ta cần chứng minh thuật toán A cho lời giải đúng. Gọi $A(I)$ là lời giải tìm được bởi thuật toán A đối với bộ dữ liệu I , còn O là lời giải tối ưu của bài toán đối với bộ dữ liệu này.
- Ta cần tìm cách xây dựng phép biến đổi φ cho phép biến đổi O thành lời giải O' sao cho
 - O' cũng tốt không kém gì O (nghĩa là O' vẫn là tối ưu);
 - O' giống với $A(I)$ nhiều hơn O .

LẬP LUẬN BIẾN ĐỔI



■ Chứng minh bằng phản chứng:

- Giả sử A không đúng đắn. Khi đó phải tìm được bộ dữ liệu I sao cho $A(I)$ khác với lời giải tối ưu của bài toán.
- Gọi O là lời giải tối ưu giống với $A(I)$ nhất. Do $A(I)$ không là lời giải tối ưu nên $A(I) \neq O$.
- Khi đó sử dụng phép biến đổi φ ta có thể biến đổi O thành O' sao cho: O' vẫn là tối ưu đồng thời O' giống với $A(I)$ hơn là O .
- Điều đó là mâu thuẫn với giả thiết O là lời giải tối ưu giống với $A(I)$ nhất.

LẬP LUẬN BIẾN ĐỔI



▪ Chứng minh trực tiếp:

- Giả sử có O là lời giải tối ưu. Khi đó ta biến đổi O thành lời giải tối ưu O' giống với $A(I)$ hơn là O .
- Nếu $O' = A(I)$ thì $A(I)$ chính là lời giải tối ưu. Ngược lại, ta lại lặp lại phép biến đổi đối với O' để thu được lời giải tối ưu O'' giống với $A(I)$ hơn là O' ,...
- Ta thu được dãy $O', O'', O''', \dots$. Mỗi phần tử của dãy này đều là phương án tối ưu, và mỗi phần tử sau lại giống với $A(I)$ hơn phần tử trước nó.
- Vì vậy dãy này phải kết thúc ở $A(I)$. Vậy $A(I)$ cũng là tối ưu.

BÀI TOÁN VỀ CÁC ĐOẠN THẲNG KHÔNG GIAO NHAU



- **Đầu vào:** Cho họ các đoạn thẳng mở:

$$C = \{(a_1, b_1), (a_2, b_2), \dots, (a_n, b_n)\}$$

- **Đầu ra:** Tập các đoạn không giao nhau có lực lượng lớn nhất
- **Ví dụ thực tế:** Bài toán xếp thời gian biểu cho các hội thảo, Bài toán lựa chọn hành động (Activity Selection), Bài toán phục vụ khách hàng trên một máy.

BÀI TOÁN VỀ CÁC ĐOẠN THẲNG KHÔNG GIAO NHAU – THUẬT TOÁN THAM LAM 1



Greedy1

- Sắp xếp các đoạn thẳng theo thứ tự tăng dần của đầu mút trái.
- Bắt đầu từ tập S là tập rỗng, ta lần lượt xét các đoạn trong danh sách theo thứ tự đã sắp xếp và bổ sung đoạn thẳng đang xét vào S nếu nó không có điểm chung với bất cứ đoạn nào trong S .

BÀI TOÁN VỀ CÁC ĐOẠN THẲNG KHÔNG GIAO NHAU – THUẬT TOÁN THAM LAM 1



procedure Greedy1;
begin

$S := \emptyset$; (* S là tập các đoạn thẳng cần tìm *)

<Sắp xếp các đoạn trong C theo thứ tự không giảm của mút trái >

while ($C \neq \emptyset$) **do**

begin

$(a_c, b_c) \leftarrow$ đoạn đầu tiên trong C;

$C := C \setminus (a_c, b_c)$;

if < (a_c, b_c) không giao với bất cứ đoạn nào trong S>

then $S := S \cup (a_c, b_c)$;

end;

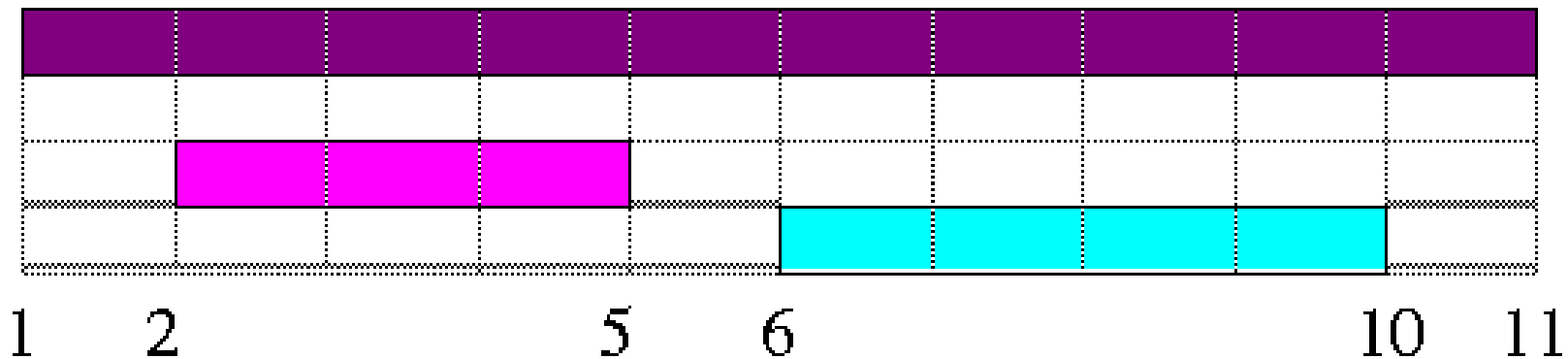
<S là tập cần tìm>

end;

BÀI TOÁN VỀ CÁC ĐOẠN THẲNG KHÔNG GIAO NHAU – THUẬT TOÁN THAM LAM



- Chứng minh tính đúng đắn: Phản ví dụ cho Greedy1

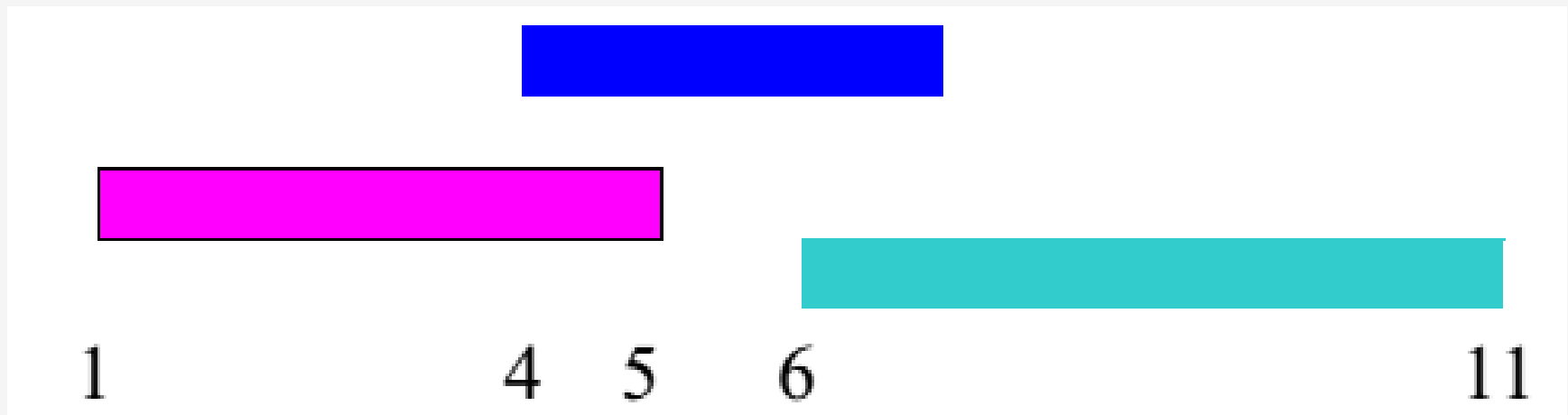


BÀI TOÁN VỀ CÁC ĐOẠN THẲNG KHÔNG GIAO NHAU – THUẬT TOÁN THAM LAM 2



Greedy2

- Ở mỗi bước ta chọn đoạn có độ dài ngắn nhất bổ sung vào S .
- Phản ví dụ: ????



BÀI TOÁN VỀ CÁC ĐOẠN THẲNG KHÔNG GIAO NHAU – THUẬT TOÁN THAM LAM 3



Greedy3

- Sắp xếp các đoạn theo thứ tự **không giảm của mút phải**.
- Bắt đầu từ tập S là tập rỗng, ta lần lượt xét các đoạn trong danh sách theo thứ tự đã sắp xếp và bổ sung đoạn thẳng đang xét vào S nếu nó không có điểm chung với bất cứ đoạn nào trong S .
- **Mệnh đề:** Thuật toán Greedy3 cho lời giải tối ưu của bài toán về các đoạn thẳng không giao nhau.

BÀI TOÁN VỀ CÁC ĐOẠN THẲNG KHÔNG GIAO NHAU – THUẬT TOÁN THAM LAM 3



Chứng minh tính đúng đắn:

- Giả sử ngược lại, thuật toán Greedy3 không cho lời giải đúng. Khi đó phải tìm được bộ dữ liệu C sao cho thuật toán không cho lời giải tối ưu. Giả sử $G_3(C)$ là lời giải tìm được bởi Greedy3. Gọi O là lời giải tối ưu có số đoạn thẳng chung với $G_3(C)$ là lớn nhất.
- Gọi X là đoạn thẳng đầu tiên trong $G_3(C)$ không có mặt trong O .
- Đoạn này là tồn tại, vì nếu trái lại thì hoặc $G_3(C) \equiv O$ (mâu thuẫn với tính không tối ưu của $G_3(C)$) hoặc $G_3(C)$ là tập con thực sự của O (trái với hoạt động của thuật toán, bởi vì khi đó thuật toán phải chọn cả đoạn cuối cùng trong O).

BÀI TOÁN VỀ CÁC ĐOẠN THẲNG KHÔNG GIAO NHAU – THUẬT TOÁN THAM LAM 3



Chứng minh tính đúng đắn (Tiếp):

- Gọi Y là đoạn đầu tiên kể từ bên trái của O không có mặt trong $G_3(C)$.

Đoạn Y phải tồn tại, vì nếu trái lại O phải là tập con của $G_3(C)$ và điều đó trái với tính tối ưu của O .

- Khi đó mút phải của đoạn X phải ở bên trái (nhỏ hơn) của mút phải của đoạn Y .

Bởi vì, nếu trái lại, thuật toán phải chọn Y thay vì chọn X .

- Xét $O' = O \setminus \{Y\} \cup \{X\}$.

BÀI TOÁN VỀ CÁC ĐOẠN THẲNG KHÔNG GIAO NHAU – THUẬT TOÁN THAM LAM 3



Chứng minh tính đúng đắn (Tiếp):

Rõ ràng:

- O' gồm các đoạn không giao nhau, bởi vì X không giao với bất cứ đoạn nào ở bên trái nó trong O' (do $G_3(C)$ là chấp nhận được) cũng như không giao với bất cứ đoạn nào ở bên phải nó trong O' (do mút phải của X nhỏ hơn mút phải của Y và Y không giao với bất cứ đoạn nào ở bên phải X trong O').
- Do O' có cùng lực lượng với O , nên O' cũng là tối ưu.
- O' giống với $G_3(C)$ hơn là O (O' có nhiều đoạn chung với $G_3(C)$ hơn là O).

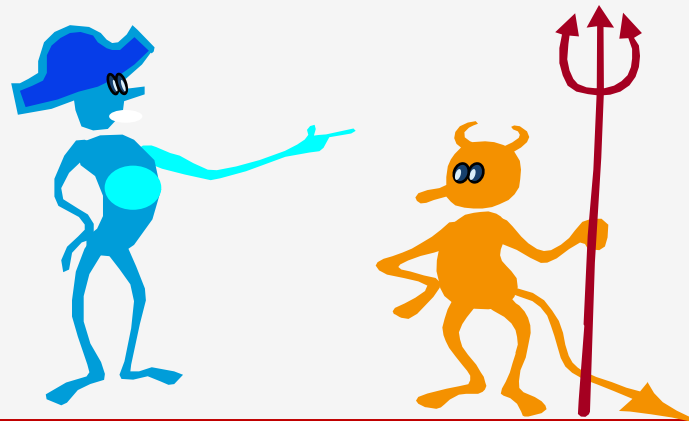
BÀI TOÁN VỀ CÁC ĐOẠN THẲNG KHÔNG GIAO NHAU – THUẬT TOÁN THAM LAM 3



Chứng minh tính đúng đắn (Tiếp):

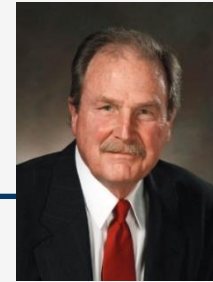
Mâu thuẫn với giả thiết là O là lời giải tối ưu giống với $G_3(C)$ nhất.

Mệnh đề được chứng minh !



MÃ HUFFMAN

David A. Huffman
1925-1999



- **Bài toán Mã hóa văn bản:** Giả sử có một văn bản trên bảng chữ cái C . Với mỗi chữ cái $c \in C$ ta biết tần suất xuất hiện của nó trong văn bản là $f(c)$. Cần tìm cách mã hóa văn bản sử dụng ít bộ nhớ nhất.
- **Mã hoá với độ dài cố định** (fixed length code). Dễ mã hoá cũng như dễ giải mã, nhưng lại đòi hỏi bộ nhớ lớn.
- **Mã phi tiền tố** (prefix free code) là cách mã hoá mỗi ký tự c bởi một xâu nhị phân $\text{code}(c)$ sao cho mã của một ký tự bất kỳ không là đoạn đầu của bất cứ mã của ký tự nào trong số các ký tự còn lại. Tuy đòi hỏi việc mã hoá và giải mã phức tạp hơn nhưng thông thường đòi hỏi ít bộ nhớ hơn.

MÃ HOÁ ĐỘ DÀI CỐ ĐỊNH



- Để mã hoá 26 chữ cái tiếng Anh bởi mã nhị phân độ dài cố định, độ dài của xâu tối thiểu là $\lceil \log 26 \rceil = 5$ bit.

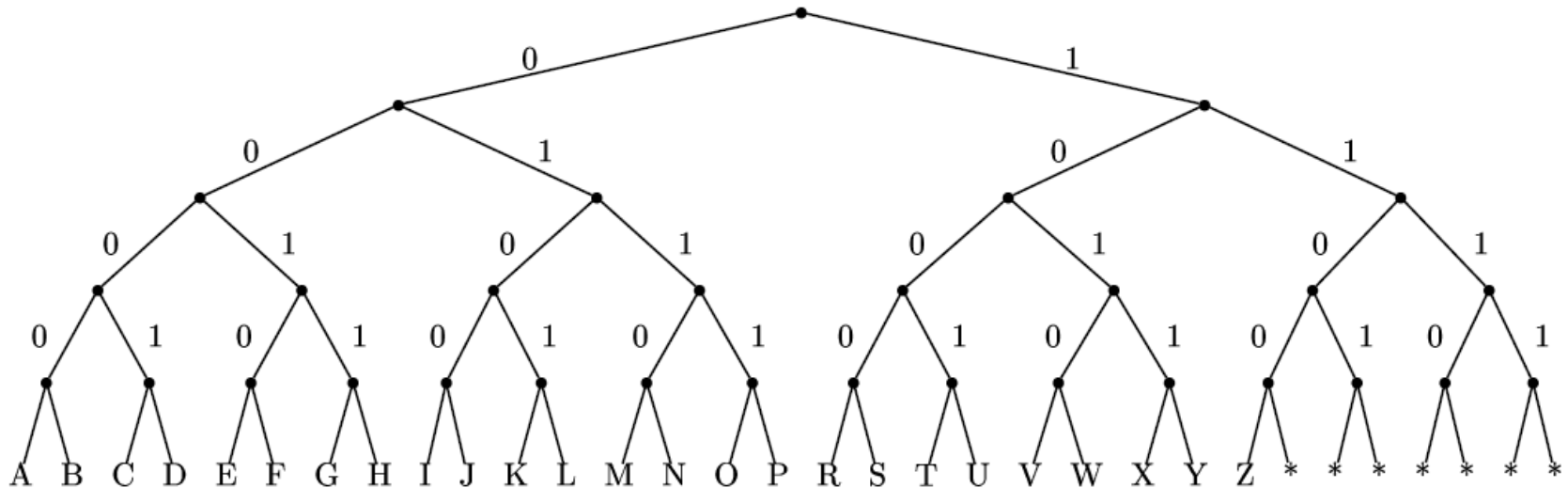
- Các xâu

00001	A	01000	H	01111	O	10110	V
00010	B	01001	I	10000	P	10111	W
00011	C	01010	J	10001	Q	11000	X
00100	D	01011	K	10010	R	11001	Y
00101	E	01100	L	10011	S	11010	Z
00110	F	01101	M	10100	T		
00111	G	01110	N	10101	U		

CÂY MÃ HOÁ ĐỘ DÀI CỐ ĐỊNH



- Mã hoá độ dài cố định là mã phi tiền tố



MORSE CODE



- Căn cứ vào **thống kê tần suất** của các chữ cái trong tiếng Anh

E	12,000
T	9,000
A, O, I, N, S	8,000
H	6,400
R	6,200
D	4,400
L	4,000
U	3,400

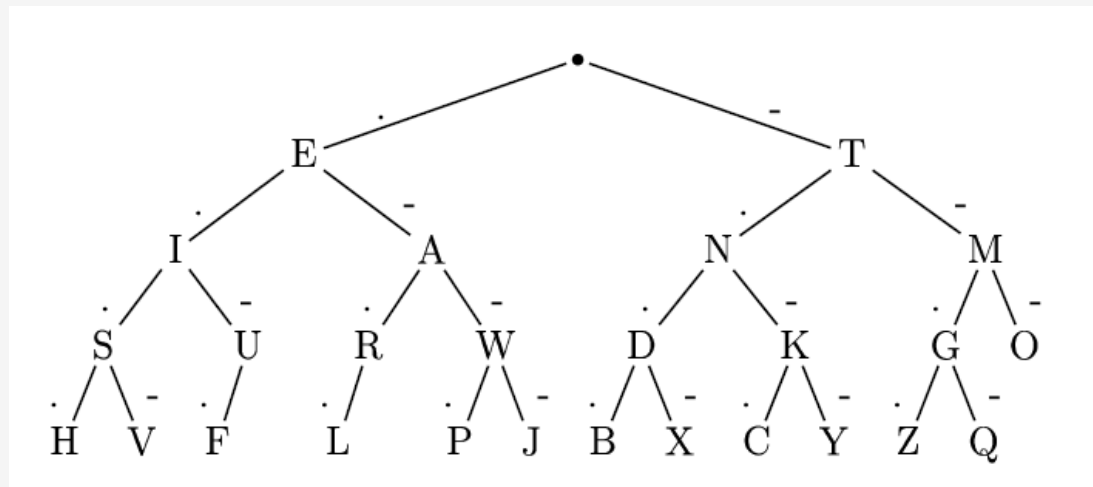
- Morse đề xuất cách mã hoá:

.	E	..	I	—	T	— —	M
.—	A	...	S	— .	N	.. —	U

CÂY MÃ HOÁ MORSE



- Mã hoá Morse **không** là phi tiền tố



- Giải mã “....-” ??? Chịu chết?
- Phải có dấu phân biệt các chữ cái: “..#..-” ➔ IU
- Cây không đầy đủ

MÃ HUFFMAN



- Mỗi mã phi tiền tố có thể biểu diễn bởi một cây nhị phân **T** mà mỗi lá của nó tương ứng với một chữ cái và cạnh của nó được gán cho một trong hai số 0 hoặc 1.
- Mã của một chữ cái **c** là 1 dãy nhị phân gồm các số gán cho các cạnh trên đường đi từ gốc đến lá tương ứng với **c**.

MÃ HUFFMAN



Bài toán: Tìm cách mã hoá tối ưu, tức là tìm cây nhị phân T làm tối thiểu hoá tổng độ dài có trọng số

$$B(T) = \sum_{c \in C} f(c) \text{ depth}(c)$$

Trong đó $\text{depth}(c)$ là độ dài đường đi từ gốc đến lá tương ứng với ký tự c .

MÃ HUFFMAN



Greedy:

- Chữ cái có tần suất nhỏ hơn cần được gán cho lá có khoảng cách đến gốc là lớn hơn;
- Chữ cái có tần suất xuất hiện lớn hơn cần được gán cho nút gần gốc hơn

MÃ HUFFMAN



```
procedure Huffman(C, f);  
begin
```

```
  n ← |C|;
```

```
  Q ← C;
```

```
  for i:=1 to n-1 do
```

```
  begin
```

```
    x, y ← 2 chữ cái có tần suất nhỏ nhất trong Q; (* Thao tác 1 *)
```

```
    Tạo nút p với hai con x, y;
```

```
    f(p) := f(x) + f(y);
```

```
    Q ← Q \ {x, y} ∪ {p} (* Thao tác 2 *)
```

```
  end;
```

```
end;
```

- Mã xây dựng theo thuật toán Huffman thường được gọi là mã Huffman (Huffman Code).
- Có thể cài đặt với thời gian $O(n \log n)$

VÍ DỤ:



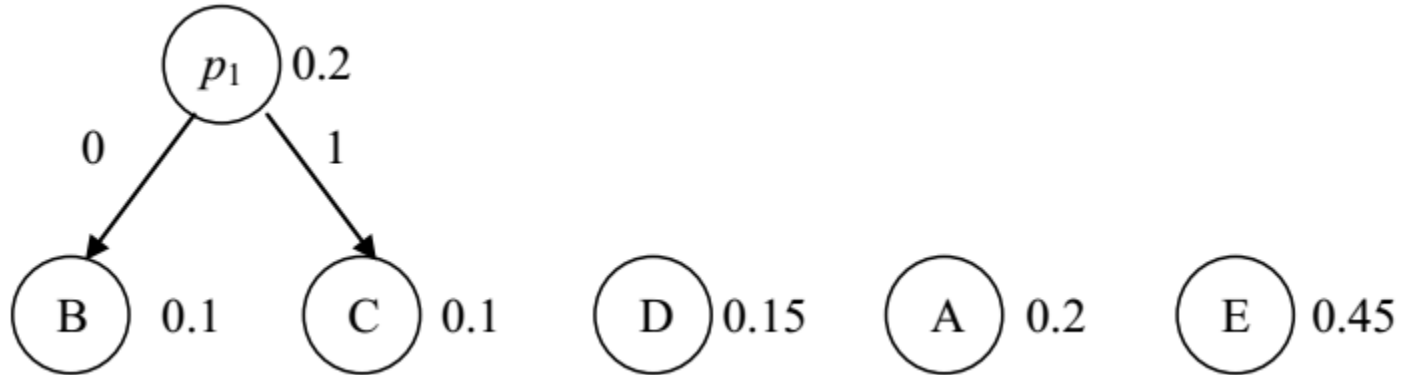
- **Ví dụ:** Giả sử văn bản cần mã hoá gồm 105 ký tự, mỗi ký tự lấy từ bảng chữ cái $\mathcal{R} = \{A, B, C, D, E\}$. Tần suất xuất hiện của các ký tự trong văn bản được cho trong bảng sau:

Kí tự	A	B	C	D	E
Xác suất	0.2	0.1	0.1	0.15	0.45

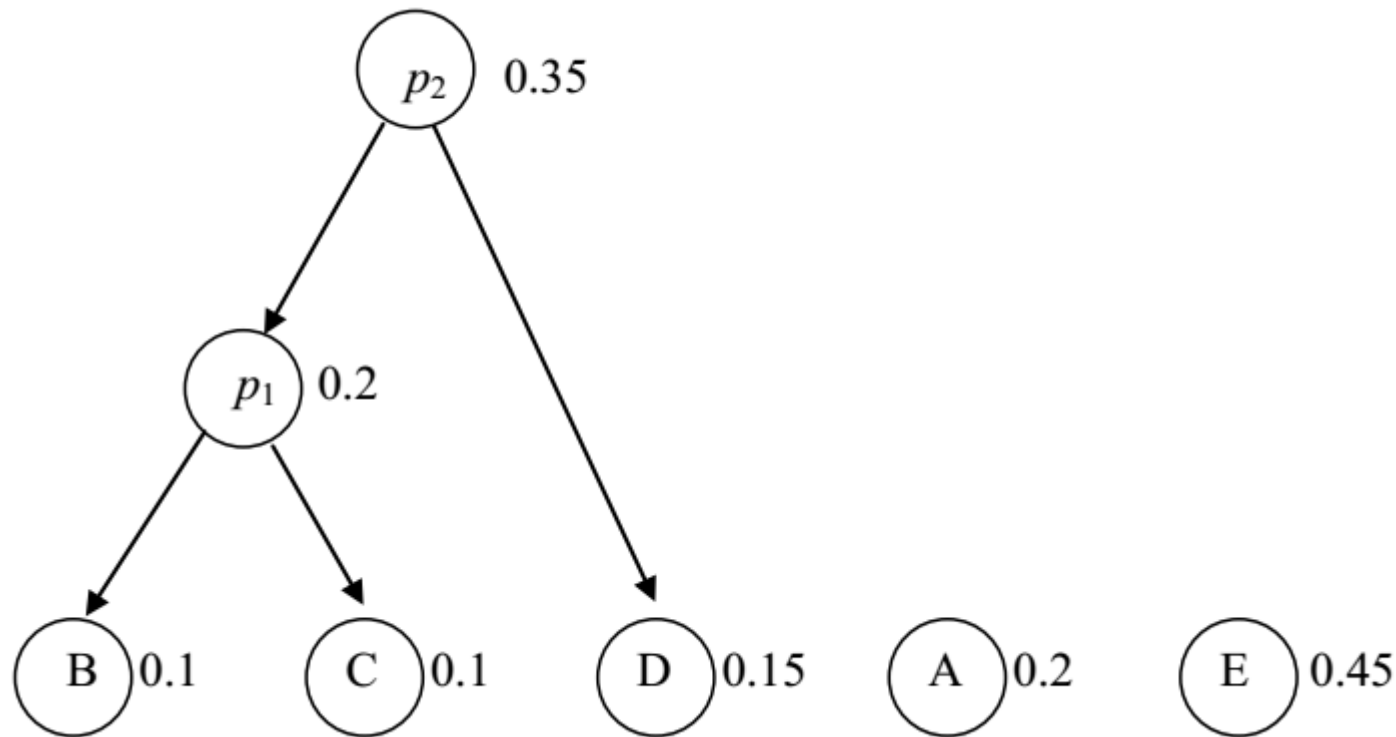
- Ta bắt đầu xây dựng mã từ tập các nút, mỗi nút ứng với một ký tự và trọng số của nút bằng tần suất của ký tự tương ứng với nó:

0.1	0.1	0.15	0.2	0.45
				

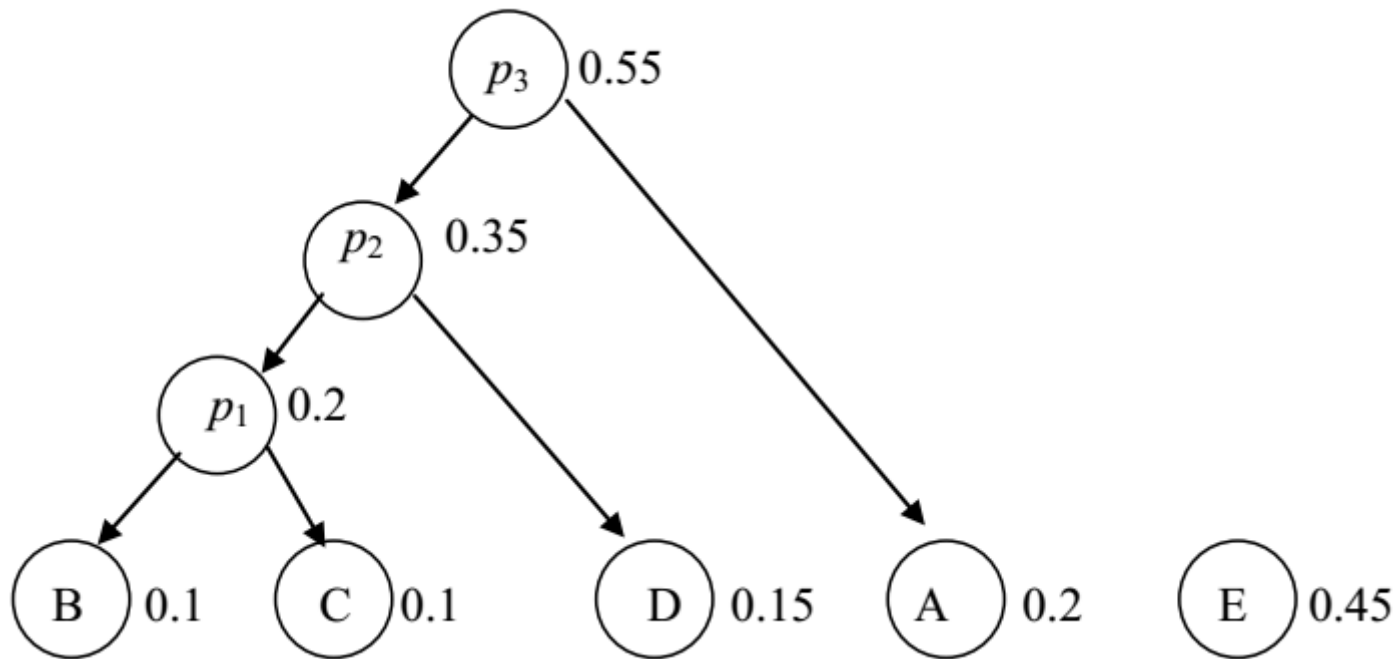
- Hai nút đầu tiên được chọn là các nút tương ứng với B và C, vì chúng có tần suất nhỏ nhất. Hai nút này được nối với một nút mới p_1 có trọng số bằng $0.1 + 0.1 = 0.2$.



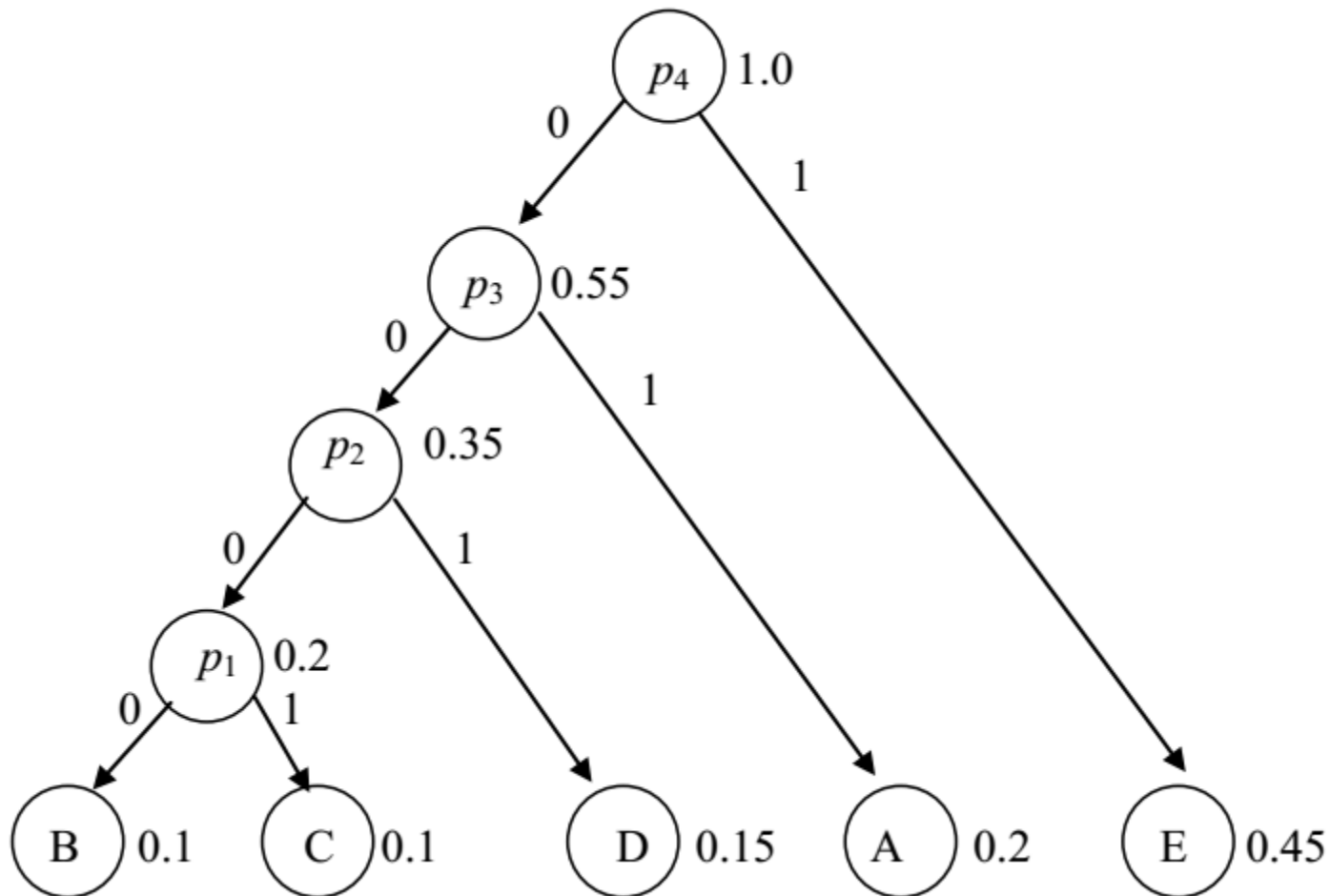
- Từ danh sách gồm 4 nút p_1 , D, A, E, ta chọn hai nút có trọng số nhỏ nhất p_1 và D (cũng có thể chọn D và A). Thay thế hai nút này bởi một nút mới p_2 có trọng số bằng $0.2 + 0.15 = 0.35$ ta thu được:



- Từ danh sách gồm các nút p_2 , A, E, ta chọn hai nút p_2 và A là hai nút có trọng số cực tiểu và tạo nút mới p_3 có trọng số bằng $0.35 + 0.2 = 0.55$ và thu được:



- Cuối cùng nối hai nút p_3 và E với nút p_4 ta thu được cây mã Huffman:



- Từ cây ta nhận được mã Huffman trong bảng sau:

Kí hiệu	Mã Huffman	Mã độ dài cố định
A	01	0000
B	0000	0001
C	0001	0010
D	001	0011
E	1	0100

MÃ HUFFMAN: GIẢI MÃ



procedure **Huffman_Decode**(B);

(* B là chuỗi mã hóa văn bản theo mã hoá Huffman. *)

begin

<Khởi động P là gốc của cây Huffman>

While <chưa đạt đến kết thúc của B> do

begin

x ← bit tiếp theo trong chuỗi B;

If x = 0 then P ← Con trái của P

Else P ← Con phải của P

If (P là nút lá) then

begin

<Hiển thị kí hiệu tương ứng với nút lá>

<Đặt lại P là gốc của cây Huffman>

end;

end

~~end;~~

VÍ DỤ:



- **Ví dụ:** Cho chuỗi $B = 010001101$ và chuỗi mã hóa thu được từ cây Huffman ở ví dụ trước. Giải mã chuỗi B để tìm văn bản ban đầu:
- Xuất phát từ gốc của cây Huffman, đi theo kí hiệu 01 được kí tự A, tương tự xét tiếp được các kí tự C, E, A.

0 1 0 0 0 1 1 0 1

A

0 1 0 0 0 1 1 0 1

A C

0 1 0 0 0 1 1 0 1

A C E

0 1 0 0 0 1 1 0 1

A C E A